Design and Architecture Document

Project: Interactive Visual Analytics for Brain Data Exploration
Project Id: 147
Guide: Dr. Leena Arya
Institution: School of Computer Science & Engineering, IILM University, Greater Noida
Stage: Phase II – Working Prototype

1. Purpose

This document describes the design and architecture of the Phase-II prototype of the Interactive Visual Analytics for Brain Data Exploration system. The prototype lets a user load multi-channel, signal-shaped data, select a channel, filter a time window, and view summary statistics and channel correlation. It is an educational prototype and is not a medical diagnostic tool.

2. Design Goals

- Make brain-signal data easy to explore interactively without writing code.
- Keep the system lightweight: it runs in a web browser with no server and no installation.
- Update every view immediately when the user changes a control.
- Validate uploaded data and give clear error messages.
- State clearly that the built-in data is synthetic.

3. System Overview

The system is a single-page, client-side web application. All data loading, processing and plotting happen in the user's browser.

Item	Detail
Application type	Single HTML page (client-side)
Languages	HTML, CSS, JavaScript
Visualization library	Plotly.js 2.35.2 (loaded from CDN)

Item	Detail
Back end / database	None
Input data	Built-in synthetic data or a user-uploaded CSV file
Output	Interactive plots, statistics, data table, exported CSV

4. Architecture

The application is organised into four layers. Each layer passes its result to the next, and any user action restarts the flow from the control layer.

Control layer → Data layer → Processing layer → Visualization layer

4.1 Data Layer

- **Synthetic data generator (`makeSample`):** creates 500 samples at 0.01 s steps (0 to 4.99 s) for four channels: Fp1, Fp2, C3 and C4. Fp1 and Fp2 are built around an 8 Hz oscillation, and C3 and C4 around a 5 Hz oscillation, each with added noise. Fp2 and C4 are phase-shifted versions of Fp1 and C3.
- **CSV parser (`parseCSV`):** reads an uploaded file, splits the header and rows, and converts values to numbers. Empty cells become missing values.
- **Validation (`setData`):** checks that the file has data rows, that a `time` column exists, and that at least one numeric signal column exists. Rows with an invalid time value are removed. Errors are shown in the Validation notes panel.

4.2 Processing Layer

- **Time filtering (`visibleRows`):** sorts the rows by time and returns the rows between the start and end slider positions, given as percentages of the data. If the start is not before the end, the end slider is corrected automatically.
- **Summary statistics (`stat`):** computes the number of valid observations, mean, standard deviation (population) and minimum and maximum for the selected channel in the visible window.
- **Correlation (`corr`):** computes the Pearson correlation coefficient between every pair of channels over the visible rows, ignoring missing values. The result is a correlation matrix.

4.3 Visualization Layer

- **Signal over time:** an interactive line chart of the selected channel, with zoom, pan and hover.
- **Channel correlation heatmap:** a colour-coded matrix from -1 to +1 with hover values.
- **Data preview table:** the first 12 rows of the visible window, with a note showing how many rows are visible.
- **Statistic cards:** visible observations, mean, standard deviation, and min/max.

4.4 Control Layer

- File upload input for a CSV file.
- Signal channel drop-down.
- Start time and end time sliders.
- Reset filters button.
- Download loaded CSV button.

The central `render()` function reads the current control values, recomputes the statistics and correlation, and redraws all views, so every view always matches the current filters.

5. Data Flow

1. The page loads and the synthetic dataset is validated and stored in memory.
2. The user may upload a CSV file, which is parsed, validated and stored instead.
3. The user selects a channel and sets the time window.
4. `visibleRows` selects the rows in the window.
5. `stat` and `corr` compute the statistics and the correlation matrix.
6. `render` updates the plot, heatmap, table and statistic cards.
7. On request, the loaded data is exported as a CSV file.

6. Data Format

Uploaded CSV files must have:

- a header row,
- a column named `time` (numeric), and

- one or more numeric signal columns, for example Fp1, Fp2, C3 and C4.

7. User Interface Layout

- **Header:** project title and a notice that this is not a diagnostic tool.
- **Left panel (Data & Controls):** upload, channel, time sliders, buttons and the synthetic-data notice.
- **Main area:** statistic cards at the top, then the signal plot, then the correlation heatmap and data preview side by side, and finally the validation notes.
- The layout is responsive and rearranges into a single column on narrow screens.

8. Validation of the Design

We tested the prototype in a browser on the synthetic dataset (500 samples, 4 channels).

Test	Visible observations	Mean	Std. dev.	Min / Max
Fp2, 60%–100% of time	201	-0.001	0.584	-0.935 / 0.922
Fp1, 72%–82% of time	52	-0.093	0.729	-1.207 / 1.195
C4, 14%–32% of time	92	-0.033	0.401	-0.703 / 0.706

- Channel selection, time filtering, statistics, plot, heatmap and data table all updated correctly together.
- The heatmap matched the way the data was generated: Fp1 and Fp2 were strongly correlated, C3 and C4 were strongly correlated, and the two pairs were close to zero (Fp1 vs C4 about -0.001).
- The signal plots showed the expected periodic oscillations.
- The CSV export button triggered a browser download request.

9. Changes from the Original Design

Original plan (Phase I)	Current prototype	Reason
Python stack: Pandas, NumPy, Matplotlib, Streamlit	Browser-based HTML, JavaScript and Plotly.js	Lighter, no setup, smoother interaction

Original plan (Phase I)	Current prototype	Reason
Public MRI, fMRI and EEG datasets	Synthetic multi-channel data first	Public data needs its source, licence and suitability checked first

10. Limitations

- Only synthetic data has been tested so far; no real public dataset has been validated.
- The CSV parser splits on commas and does not handle quoted fields.
- Plotly.js is loaded from a CDN, so an internet connection is needed on first load.
- Performance on very large files has not been measured.
- The prototype handles signal-type (time-series) data only. Brain-region, anatomical and connectivity views are not yet included.

11. Future Work

- Test with public EEG and fMRI datasets after checking their licence and provenance.
- Add brain-region and connectivity visualizations.
- Improve CSV parsing and add support for larger files.
- Add more analysis options, such as frequency-band views.